Computer Science Investigatory Project

NAME: Shaafiq N

CLASS: 12th 'A'

SUBJECT: Computer Science

TOPIC: Mini Super Market Application

Signature of the Subject teacher Signature of the Student

Acknowledgement

I, **Shaafiq N**, would like to express my sincere gratitude to my Computer Science teacher for the valuable guidance, encouragement, and support provided during the completion of this investigatory project on **Mini Super Market Application**. The guidance and suggestions helped me understand the topic in a better and more meaningful way.

I would also like to thank my school for giving me the opportunity to work on this project and enhance my knowledge through research and analysis. My heartfelt thanks go to my parents and friends for their constant support, motivation, and cooperation throughout the project.

Lastly, I extend my gratitude to everyone who directly or indirectly contributed to the successful completion of this investigatory project.

Certificate

This is to certify that **Shaafiq N** has successfully completed the Computer Science Investigatory Project on the topic **"Mini Super Market Application"** during the academic year **2026–27** under the guidance of the English teacher.

This project is a genuine piece of work carried out by the student as part of the academic requirements. The project reflects sincere effort, research, and understanding of the topic.

Teacher's Signature: _____

Student's Signature: _____

Date: _____

Index

❖INTRODUCTION

The Super Market Management System is a Python-based application developed to simplify product management in a supermarket. It is connected to a MySQL database that stores and manages product information efficiently. The system provides functions such as adding products, removing products, updating product details, searching for products, checking expired products, viewing available stock, and generating customer bills.

The primary objective and basic agenda of this project is to ensure customer safety by clearly displaying in the generated bill whether a product is expired, not expired, or has no expiry date. This helps customers and shopkeepers verify that the product being sold is safe for consumption or use.

The application automatically checks product expiry dates and displays their status during billing. This helps prevent the sale of expired products and improves inventory management.

The project has been developed using Python and MySQL. Although the current version uses a text-based interface, it can also be implemented as a Graphical User Interface (GUI) with tools such as Tkinter, PyQt, or CustomTkinter. A GUI version would provide a more user-friendly and professional experience while maintaining the same functionality.

Overall, this project demonstrates the practical application of programming and database management in supermarket operations.

❖Database for the Super market application

```
mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| sakila             |
| super_market       |
| sys                |
| world              |
+--------------------+
7 rows in set (0.03 sec)

mysql> show tables from Super_Market;
+------------------------+
| Tables_in_super_market |
+------------------------+
| products               |
+------------------------+
1 row in set (0.02 sec)

mysql> use Super_Market;
Database changed
mysql> select * from Products;
+------------+--------------+--------------------------+--------------------------+-------------+
| Product_ID | Product_Name | Total_Quantity_Available | Individual_Product_Price | Expiry_Date |
+------------+--------------+--------------------------+--------------------------+-------------+
|          1 | Chocolate    |                       70 |                    50.00 | 2027-05-09  |
|          2 | Milk         |                       20 |                    45.50 | 2026-05-08  |
|          3 | Bread        |                       15 |                    30.00 | 2026-05-08  |
|          4 | Biscuits     |                       50 |                    20.00 | 2026-08-20  |
|          5 | Bottle       |                       30 |                   150.00 | NULL        |
|          6 | Plates       |                       40 |                    70.00 | NULL        |
|          7 | Toot Brush   |                       60 |                    40.00 | NULL        |
|          8 | Toot Paste   |                       60 |                    10.00 | NULL        |
|          9 | Cake         |                       40 |                    20.00 | 2026-09-26  |
|         10 | Juice        |                       20 |                    20.00 | 2026-10-02  |
|         11 | Pencil Set   |                        0 |                    20.00 | NULL        |
|         12 | Glue         |                        5 |                    10.00 | NULL        |
|         13 | Butter       |                      100 |                    60.00 | 2026-09-06  |
|         14 | Books        |                       40 |                   100.00 | NULL        |
+------------+--------------+--------------------------+--------------------------+-------------+
14 rows in set (0.00 sec)
```

The above picture displays the MySQL database used in the project. It shows the list of available databases, including the **Super_Market** database created specifically for this application. The picture also shows the **Products** table stored within the database. The table contains details of all products available in the supermarket, including Product ID, Product Name, Total Quantity Available, Individual Product Price, and Expiry Date. This data is used by the Python program to perform operations such as billing, product searching, stock management, expiry checking, and product updates. The records

displayed confirm that the database has been successfully populated with sample product information for testing and implementation purposes.

```
mysql> describe products;
+--------------------------+----------------------+------+-----+---------+----------------+
| Field                    | Type                 | Null | Key | Default | Extra          |
+--------------------------+----------------------+------+-----+---------+----------------+
| Product_ID               | bigint unsigned      | NO   | PRI | NULL    | auto_increment |
| Product_Name             | char(50)             | NO   | PRI | NULL    |                |
| Total_Quantity_Available | bigint unsigned      | YES  |     | NULL    |                |
| Individual_Product_Price | decimal(5,2) unsigned| NO   |     | NULL    |                |
| Expiry_Date              | date                 | YES  |     | NULL    |                |
+--------------------------+----------------------+------+-----+---------+----------------+
5 rows in set (0.01 sec)
```

The above picture displays the structure of the **Products** table using the DESCRIBE Products; command in MySQL. It shows the fields, data types, constraints, and attributes associated with each column. The **Product_ID** field is defined as an auto-incrementing primary key, ensuring a unique identification number for every product. The **Product_Name** field stores the name of the product, while **Total_Quantity_Available** stores the stock quantity. The **Individual_Product_Price** field stores the price of a single unit of the product, and the **Expiry_Date** field stores the product's expiry date. This table structure forms the foundation of the Super Market Management System and enables efficient storage, retrieval, and management of product information.

❖Program or Code for the Super market application

```
import mysql.connector
from datetime import date,datetime


while True:
    try:
        db=mysql.connector.connect(
            host="localhost",
            user="root",
            password="shafi_@62(",
            database="Super_Market")


        c=db.cursor()
        break
    except:
        print("Unable to connect database")
        while True:
            try:
                a=input("Do you want to try again('y','n'): ").lower()
            except:
                print("Please enter 'y' or 'n'\n")
                continue
            if a!='y' and a!='n':
                print("Please enter 'y' or 'n'\n")
                continue
            else:
                break
        if a=='n':
```

```
            exit()
        else:
            print("\n")


print("SUPER MARKET APPLICATION\n")


def expiry(x):
    c.execute("select Expiry_Date from products where Product_ID=%s",(x,))
    d=c.fetchone()
    if d and d[0] is not None:
        if d[0]>date.today():
            print("Not expired")
        elif d[0]<=date.today():
            print("Expired")
    else:
        print("No expiry date")


def add_to_list(d,z,q,p):
    if d[2] == 0:
        print("\nProduct out of stock")
    elif z>d[2]:
        print("\nOnly", d[2], "items available")
        z=d[2]
        p.append([d,z])
        print(f"{'ID':>5} | {'Name':<13} | {'Price':>11} | {'Quantity':>8} |
{'E.C':<13}")
        print("="*63)
        print(f"{d[0]:>5} | {d[1]:<13} | {d[3]:>11} | {z:>8} | ",end='')
        expiry(q)
    else:
```

```
        p.append([d,z])
        print(f"{'ID':>5} | {'Name':<13} | {'Price':>11} | {'Quantity':>8} |
{'E.C':<13}")
        print("="*63)
        print(f"{d[0]:>5} | {d[1]:<13} | {d[3]:>11} | {z:>8} | ",end='')
        expiry(q)


def billing_system():
    p=[]
    t=0
    while True:
        while True:
            try:
                q=int(input("\nEnter the product ID: "))
                break
            except:
                print("Please enter a proper ID")
                continue


        while True:
            try:
                z=int(input("Enter the quantity: "))
            except:
                print("Please enter a proper quantity\n")
                continue
            if z<=0:
                print("Quantity must be greater than 0")
                continue
            else:
                break
```

```
        c.execute("select * from Products where Product_ID=%s",(q,))
        d=c.fetchone()
        if d is not None:
            if d[0] in (i[0][0] for i in p):
                print("Product already added to the list")
                while True:
                    while True:
                        try:
                            a=input("\nDo you want to increase the quantity of this
product(y,n): ").lower()
                            break
                        except:
                            print("Please enter 'y' or 'n'")
                    if a!='y' and a!='n':
                        print("Please enter 'y' or 'n'")
                        continue
                    else:
                        break
                if a=='y':
                    for k in p:
                        if k[0][0] == d[0]:
                            if k[1]+z >d[2]:
                                print("\nOnly", d[2]-k[1], "more items available")
                                k[1]=d[2]
                            else:
                                k[1] += z
                            break
                else:
                    print("Quantity not updated")
```

```
            else:
                add_to_list(d,z,q,p)
        else:
            print("\nProduct not available")
        while True:
            while True:
                try:
                    a=input("\n\nDo you want to add another product(y,n): ").lower()
                    break
                except:
                    print("Please enter 'y' or 'n'")
            if a!='y' and a!='n':
                print("Please enter 'y' or 'n'")
                continue
            else:
                break
        if a=='y':
            continue
        else:
            print("\nFinal bill is:\n")
            print(f"{'ID':>5} | {'Name':<13} | {'Price':>12} | {'Quantity':>8} |
{'E.C':<13}")
            print("="*64)
            for i,z in p:
                print(f"{i[0]:>5} | {i[1]:<13} | {i[3]:>12} | {z:>8} | ",end='')
                expiry(i[0])

                t+=i[3]*z
                current_quantity = i[2] - z
                c.execute("update Products set Total_Quantity_Available=%s where
Product_ID=%s",(current_quantity, i[0]))
```

```
            db.commit()
            print("\nTotal Price:",t)
            p.clear()
            t=0
            print("\n")
            break


def add_product():
    while True:
        while True:
            try:
                n=input("\nEnter the product name: ").strip().title()
            except:
                print("Please enter a proper name")
                continue
            if n=="":
                print("Product name cannot be empty")
                continue
            else:
                break
        c.execute("select * from Products where Product_Name=%s",(n,))
        d=c.fetchone()
        if d is not None:
            print("Product already available\n")
            while True:
                while True:
                    try:
                        f=input("Do you want to increase the product's quantity
instead('y','n'): ").lower()
                        break
```

```
                    except:
                        print("Please enter 'y' or 'n'")
                        continue
                if f!='y' and f!='n':
                    print("Please enter 'y' or 'n'\n")
                    continue
                else:
                    break
            if f=='y':
                while True:
                    try:
                        t=int(input("\nEnter the total quantity: "))
                    except:
                        print("Please enter a proper quantity")
                        continue
                    if t<0:
                        print("Quantity must not be in negative")
                        continue
                    else:
                        break
                c.execute("update products set Total_Quantity_Available=%s where
Product_Name=%s",(d[2]+t,n))
                db.commit()
                print("\nProduct's Quantity has be successfully Increased\n")
            else:
                print("\n")
        else:
            while True:
                try:
                    t=int(input("Enter the total quantity: "))
```

```
            except:
                print("Please enter a proper quantity\n")
                continue
            if t<0:
                print("Quantity must not be in negative")
                continue
            else:
                break
        while True:
            try:
                p=float(input("Enter the individual product price: "))
            except:
                print("Please enter a proper price value\n")
                continue
            if p<=0:
                print("Price must be greater than 0")
                continue
            else:
                break
        while True:
            try:
                e=input("Enter the expiry date of the product(YYYY-MM-DD): ")
                if e=="":
                    e=None
                    break
                e=datetime.strptime(e,"%Y-%m-%d").date()
                break
            except:
                print("Invalid date format\n")
                continue
```

```
            c.execute("insert into Products(Product_Name, Total_Quantity_Available,
Individual_Product_Price, Expiry_Date) values(%s,%s,%s,%s)",(n,t,p,e))
            db.commit()
            print("\nProduct",n,"successfully added to the database of the Super
Market\n\n")
        while True:
            while True:
                try:
                    a=input("Do you want to add another product(y,n): ").lower()
                    break
                except:
                    print("Please enter 'y' or 'n'")
            if a!='y' and a!='n':
                print("Please enter 'y' or 'n'\n")
                continue
            else:
                break
        if a=='y':
            continue
        else:
            print("\n")
            break


def remove_product():
    while True:
        while True:
            try:
                r=input("\nEnter the product name or ID to be removed:
").strip().title()
                break
            except:
```

```
                print("Please enter a proper ID or name")
        if r.isdigit():
            c.execute("select * from Products where Product_ID=%s",(r,))
            d=c.fetchone()
            if d:
                c.execute("delete from Products where Product_ID=%s",(r,))
                print("\nProduct with ID",r,"is successfully removed from the
database of the Super Market\n\n")
            else:
                print("Product not available\n")
        else:
            c.execute("select * from Products where Product_Name=%s",(r,))
            d=c.fetchone()
            if d:
                c.execute("delete from Products where Product_Name=%s",(r,))
                print("\nProduct with name",r,"is successfully removed from the
database of the Super Market\n\n")
            else:
                print("Product not available\n")
        db.commit()
        while True:
            while True:
                try:
                    a=input("Do you want to remove another product(y,n): ").lower()
                    break
                except:
                    print("Please enter 'y' or 'n'")
            if a!='y' and a!='n':
                print("Please enter 'y' or 'n'\n")
                continue
            else:
```

```
                break
        if a=='y':
            continue
        else:
            print("\n")
            break


def search_product():
    while True:
        try:
            s=input("\nEnter the product name or ID to search for it:
").strip().title()
            break
        except:
            print("Please enter a proper ID or name")
    if s.isdigit():
        c.execute("select * from Products where Product_ID=%s",(s,))
    else:
        c.execute("select * from Products where Product_Name=%s",(s,))
    d=c.fetchone()
    if d is not None:
        print("\n")
        print(f"{'ID':>5} | {'Name':<13} | {'Total Quantity':<14} | {'Price':<11} |
{'Expiry Date':<13}")
        print("="*66)
        if d[4] is not None:
            expiry_date =d[4]
        else:
            expiry_date ="N/A"
        print(f"{d[0]:>5} | {d[1]:<13} | {d[2]:<14} | {d[3]:<11} |
{expiry_date:<13}")
```

```
        print("\n")
    else:
        print("Product not available\n")


def expiry_checker():
    c.execute("select * from products where Expiry_Date<=curdate()")
    d=c.fetchall()
    if d:
        print("\n")
        print(f"{'ID':>5} | {'Name':<13} | {'Total Quantity':>14} | {'Price':>11} |
{'Expiry Date':<13} | {'E.C':<15}")
        print("="*78)
        for i in d:
            expiry_date=str(i[4])
            print(f"{i[0]:>5} | {i[1]:<13} | {i[2]:>14} | {i[3]:>11} |
{expiry_date:<13} | {'Expired':<15}")
        print("\n")
    else:
        print("\nNo expired products\n")


def view_products():
    c.execute("select * from products")
    d=c.fetchall()
    print("\n")
    print(f"{'ID':>5} | {'Name':<13} | {'Total Quantity':>14} | {'Price':>11} |
{'Expiry Date':<13}")
    print("="*66)
    if d:
        for i in d:
            if i[4] is not None:
                expiry_date =str(i[4])
```

```
            else:
                expiry_date ="N/A"
            print(f"{i[0]:>5} | {i[1]:<13} | {i[2]:>14} | {i[3]:>11} |
{expiry_date:<13}")
        print("\n")
    else:
        print("\nNo products\n")


def product_data_update():
    while True:
        while True:
            try:
                v=input("\nEnter the product ID or name to be updated in the database
of the Super Market: ").strip().title()
                break
            except:
                print("Please enter a proper ID or name")
        if v.isdigit():
            c.execute("select * from Products where Product_ID=%s",(v,))
        else:
            c.execute("select * from Products where Product_Name=%s",(v,))
        d=c.fetchone()
        if d is not None:
            print("\n1= Total quantity available")
            print("2= Individual product price")
            print("3= Expiry date\n")
            while True:
                try:
                    p=int(input("Enter the integer of the data need to be updated for
the product in the database of the Super Market: "))
                except:
```

```
                print("Please enter the proper integer\n")
                continue
            n=range(1,4)
            if p not in n:
                print("Please enter a proper integer from the given option\n")
                continue
            else:
                break
        if p==1:
            while True:
                try:
                    q=int(input("\nEnter the quantity to be be updated: "))
                except:
                    print("Please enter a proper quantity")
                    continue
                if q<0:
                    print("Quantity must not be negative")
                    continue
                else:
                    break
            c.execute("update products set Total_Quantity_Available=%s where
Product_ID=%s",(q,d[0]))
            print("\nQuantity of the product",d[1],"has been successfully
updated")
        elif p==2:
            while True:
                try:
                    q=float(input("\nEnter the price to be be updated: "))
                except:
                    print("Please enter a proper price value")
                    continue
```

```
                    if q<=0:
                        print("Price must be greater than 0")
                        continue
                    else:
                        break
                c.execute("update products set Individual_Product_Price=%s where
Product_ID=%s",(q,d[0]))
                print("\nIndividual product price of the product",d[1],"has been
successfully updated")
            elif p==3:
                while True:
                    try:
                        q=input("\nEnter the expiry date of the product(YYYY-MM-DD):
")
                        if q=="":
                            q=None
                            break
                        q=datetime.strptime(q,"%Y-%m-%d").date()
                        break
                    except:
                        print("Invalid date format\n")
                        continue
                c.execute("update products set Expiry_Date=%s where
Product_ID=%s",(q,d[0]))
                print("\nExpiry date of the product",d[1],"has been successfully
updated")
            db.commit()
        else:
            print("Product not available\n")
        while True:
            while True:
                try:
```

```
                    a=input("\nDo you want to update another product(y,n): ").lower()
                    break
                except:
                    print("Please enter 'y' or 'n'\n")
            if a!='y' and a!='n':
                print("Please enter 'y' or 'n'\n")
                continue
            else:
                break
        if a=='y':
            continue
        else:
            print("\n")
            break

k=range(1,9)

while True:
    print("1= Billing")
    print("2= Add products")
    print("3= Remove product")
    print("4= Search products")
    print("5= Show expired products")
    print("6= Show all available products")
    print("7= Update product data")
    print("8= Exit Super Market\n")
    try:
        l=int(input("Which Super Market operation do you want to perform(Enter the
integer): "))
    except:
```

```
        print("Please enter the proper integer\n")
        continue
    if l not in k:
        print("Please enter the integer from the given data\n")
        continue
    if l==1:
        billing_system()
    elif l==2:
        add_product()
    elif l==3:
        remove_product()
    elif l==4:
        search_product()
    elif l==5:
        expiry_checker()
    elif l==6:
        view_products()
    elif l==7:
        product_data_update()
    elif l==8:
        print("\nThank you for using the Super Market Application")
        break
c.close()
db.close()
```

❖OUTPUT

```
SUPER MARKET APPLICATION

1= Billing
2= Add products
3= Remove product
4= Search products
5= Show expired products
6= Show all available products
7= Update product data
8= Exit Super Market

Which Super Market operation do you want to perform(Enter the integer):
```

This picture shows the main menu of the Super Market Application as displayed in Python IDLE. It presents the various operations available to the user, such as billing, adding and removing products, searching products, viewing stock details, checking expired products, updating product data, and exiting the application.

```
Which Super Market operation do you want to perform(Enter the integer): 6


   ID | Name          | Total Quantity |       Price | Expiry Date
=================================================================
    1 | Chocolate     |             70 |       50.00 | 2027-05-09
    2 | Milk          |             20 |       45.50 | 2026-05-08
    3 | Bread         |             15 |       30.00 | 2026-05-08
    4 | Biscuits      |             50 |       20.00 | 2026-08-20
    5 | Bottle        |             30 |      150.00 | N/A
    6 | Plates        |             40 |       70.00 | N/A
    7 | Toot Brush    |             60 |       40.00 | N/A
    8 | Toot Paste    |             60 |       10.00 | N/A
    9 | Cake          |             40 |       20.00 | 2026-09-26
   10 | Juice         |             20 |       20.00 | 2026-10-02
   11 | Pencil Set    |              0 |       20.00 | N/A
   12 | Glue          |              5 |       10.00 | N/A
   13 | Butter        |            100 |       60.00 | 2026-09-06
   14 | Books         |             40 |      100.00 | N/A
```

This picture shows the “Show All Available Products” feature of the Super Market Application as displayed in Python IDLE. It presents a complete list of products stored in the database along with their Product ID, Product Name, Available Quantity, Individual Price, and Expiry Date, allowing the user to view and monitor the current inventory of the supermarket.

```
Which Super Market operation do you want to perform(Enter the integer): 5


   ID | Name          | Total Quantity |       Price | Expiry Date   | E.C
=================================================================================
    2 | Milk          |             20 |       45.50 | 2026-05-08    | Expired
    3 | Bread         |             15 |       30.00 | 2026-05-08    | Expired
```

This picture shows the “Show Expired Products” feature of the Super Market Application as displayed in Python IDLE. It lists all products whose expiry dates have passed, along with their Product ID, Product Name, Available Quantity, Price, Expiry Date, and Expiry Condition, helping the user identify products that are no longer safe for sale.

```
Which Super Market operation do you want to perform(Enter the integer): 4

Enter the product name or ID to search for it: 5


   ID | Name          | Total Quantity | Price       | Expiry Date
=================================================================
    5 | Bottle        | 30             | 150.00      | N/A
```

```
Which Super Market operation do you want to perform(Enter the integer): 4

Enter the product name or ID to search for it: Bottle


   ID | Name          | Total Quantity | Price       | Expiry Date
=================================================================
    5 | Bottle        | 30             | 150.00      | N/A
```

This picture shows the “Search Product” feature of the Super Market Application as displayed in Python IDLE. It displays the details of a specific product searched by the user, including its Product ID, Product Name, Available Quantity, Price, and Expiry Date, allowing quick retrieval of product information from the database.

```
Which Super Market operation do you want to perform(Enter the integer): 2

Enter the product name: Soap
Enter the total quantity: 30
Enter the individual product price: 60
Enter the expiry date of the product(YYYY-MM-DD): 2027-06-01

Product Soap successfully added to the database of the Super Market


Do you want to add another product(y,n): y

Enter the product name: Soap
Product already available

Do you want to increase the product's quantity instead('y','n'): y

Enter the total quantity: 5

Product's Quantity has be successfully Increased
```

```
mysql> select * from Products;
+------------+--------------+--------------------------+--------------------------+-------------+
| Product_ID | Product_Name | Total_Quantity_Available | Individual_Product_Price | Expiry_Date |
+------------+--------------+--------------------------+--------------------------+-------------+
|          1 | Chocolate    |                       70 |                    50.00 | 2027-05-09  |
|          2 | Milk         |                       20 |                    45.50 | 2026-05-08  |
|          3 | Bread        |                       15 |                    30.00 | 2026-05-08  |
|          4 | Biscuits     |                       50 |                    20.00 | 2026-08-20  |
|          5 | Bottle       |                       30 |                   150.00 | NULL        |
|          6 | Plates       |                       40 |                    70.00 | NULL        |
|          7 | Toot Brush   |                       60 |                    40.00 | NULL        |
|          8 | Toot Paste   |                       60 |                    10.00 | NULL        |
|          9 | Cake         |                       40 |                    20.00 | 2026-09-26  |
|         10 | Juice        |                       20 |                    20.00 | 2026-10-02  |
|         11 | Pencil Set   |                        0 |                    20.00 | NULL        |
|         12 | Glue         |                        5 |                    10.00 | NULL        |
|         13 | Butter       |                      100 |                    60.00 | 2026-09-06  |
|         14 | Books        |                       40 |                   100.00 | NULL        |
|         15 | Soap         |                       35 |                    60.00 | 2027-06-01  |
+------------+--------------+--------------------------+--------------------------+-------------+
15 rows in set (0.00 sec)
```

Database after adding the product “Soap” and after increasing its quantity

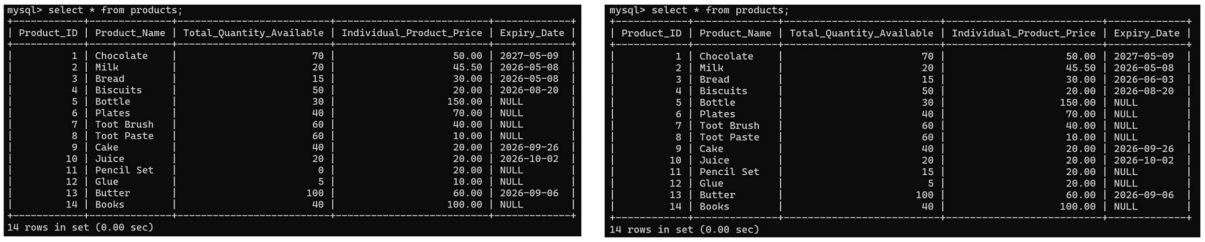
The output shows the product addition feature of the Super Market Management System. A new product named "Soap" is added successfully to the database. When the same product is entered again, the system detects the duplicate entry and offers to increase its quantity. The stock quantity is then updated successfully.

```
Which Super Market operation do you want to perform(Enter the integer): 3
Enter the product name or ID to be removed: 15

Product with ID 15 is successfully removed from the database of the Super Market
```

```
mysql> select * from Products;
+------------+--------------+-------------------------+--------------------------+-------------+
| Product_ID | Product_Name | Total_Quantity_Available | Individual_Product_Price | Expiry_Date |
+------------+--------------+-------------------------+--------------------------+-------------+
|          1 | Chocolate    |                      70 |                    50.00 | 2027-05-09  |
|          2 | Milk         |                      20 |                    45.50 | 2026-05-08  |
|          3 | Bread        |                      15 |                    30.00 | 2026-05-08  |
|          4 | Biscuits     |                      50 |                    20.00 | 2026-08-20  |
|          5 | Bottle       |                      30 |                   150.00 | NULL        |
|          6 | Plates       |                      40 |                    70.00 | NULL        |
|          7 | Toot Brush   |                      60 |                    40.00 | NULL        |
|          8 | Toot Paste   |                      60 |                    10.00 | NULL        |
|          9 | Cake         |                      40 |                    20.00 | 2026-09-26  |
|         10 | Juice        |                      20 |                    20.00 | 2026-10-02  |
|         11 | Pencil Set   |                       0 |                    20.00 | NULL        |
|         12 | Glue         |                       5 |                    10.00 | NULL        |
|         13 | Butter       |                     100 |                    60.00 | 2026-09-06  |
|         14 | Books        |                      40 |                   100.00 | NULL        |
+------------+--------------+-------------------------+--------------------------+-------------+
14 rows in set (0.01 sec)
```

The output demonstrates the product removal feature of the Super Market Management System. The user selects the delete operation and enters Product ID 15. The program searches the database, removes the corresponding product record, and displays a confirmation message. This ensures accurate inventory management and efficient database maintenance.

```
Which Super Market operation do you want to perform(Enter the integer): 7

Enter the product ID or name to be updated in the database of the Super Market: 11

1= Total quantity available
2= Individual product price
3= Expiry date

Enter the integer of the data need to be updated for the product in the database of the Super Market: 1

Enter the quantity to be be updated: 15

Quantity of the product Pencil Set has been successfully updated

Do you want to update another product(y,n): y

Enter the product ID or name to be updated in the database of the Super Market: 12

1= Total quantity available
2= Individual product price
3= Expiry date

Enter the integer of the data need to be updated for the product in the database of the Super Market: 2

Enter the price to be be updated: 20

Individual product price of the product Glue has been successfully updated

Do you want to update another product(y,n): y

Enter the product ID or name to be updated in the database of the Super Market: 3

1= Total quantity available
2= Individual product price
3= Expiry date

Enter the integer of the data need to be updated for the product in the database of the Super Market: 3

Enter the expiry date of the product(YYYY-MM-DD): 2026-06-03

Expiry date of the product Bread has been successfully updated
```



```
mysql> select * from products;
+------------+--------------+--------------------------+---------------------------+-------------+
| Product_ID | Product_Name | Total_Quantity_Available | Individual_Product_Price  | Expiry_Date |
+------------+--------------+--------------------------+---------------------------+-------------+
|          1 | Chocolate    |                       70 |                     50.00 | 2027-05-09  |
|          2 | Milk         |                       20 |                     45.50 | 2026-05-08  |
|          3 | Bread        |                       15 |                     30.00 | 2026-05-08  |
|          4 | Biscuits     |                       50 |                     20.00 | 2026-08-20  |
|          5 | Bottle       |                       30 |                    150.00 | NULL        |
|          6 | Plates       |                       40 |                     70.00 | NULL        |
|          7 | Toot Brush   |                       60 |                     40.00 | NULL        |
|          8 | Toot Paste   |                       60 |                     10.00 | NULL        |
|          9 | Cake         |                       40 |                     20.00 | 2026-09-26  |
|         10 | Juice        |                       20 |                     20.00 | 2026-10-02  |
|         11 | Pencil Set   |                        0 |                     20.00 | NULL        |
|         12 | Glue         |                        5 |                     10.00 | NULL        |
|         13 | Butter       |                      100 |                     60.00 | 2026-09-06  |
|         14 | Books        |                       40 |                    100.00 | NULL        |
+------------+--------------+--------------------------+---------------------------+-------------+
14 rows in set (0.00 sec)

mysql> select * from products;
+------------+--------------+--------------------------+---------------------------+-------------+
| Product_ID | Product_Name | Total_Quantity_Available | Individual_Product_Price  | Expiry_Date |
+------------+--------------+--------------------------+---------------------------+-------------+
|          1 | Chocolate    |                       70 |                     50.00 | 2027-05-09  |
|          2 | Milk         |                       20 |                     45.50 | 2026-05-08  |
|          3 | Bread        |                       15 |                     30.00 | 2026-06-03  |
|          4 | Biscuits     |                       50 |                     20.00 | 2026-08-20  |
|          5 | Bottle       |                       30 |                    150.00 | NULL        |
|          6 | Plates       |                       40 |                     70.00 | NULL        |
|          7 | Toot Brush   |                       60 |                     40.00 | NULL        |
|          8 | Toot Paste   |                       60 |                     10.00 | NULL        |
|          9 | Cake         |                       40 |                     20.00 | 2026-09-26  |
|         10 | Juice        |                       20 |                     20.00 | 2026-10-02  |
|         11 | Pencil Set   |                       15 |                     20.00 | NULL        |
|         12 | Glue         |                        5 |                     20.00 | NULL        |
|         13 | Butter       |                      100 |                     60.00 | 2026-09-06  |
|         14 | Books        |                       40 |                    100.00 | NULL        |
+------------+--------------+--------------------------+---------------------------+-------------+
14 rows in set (0.00 sec)
```

Before and After Updating the Products

The output demonstrates the product update feature of the Super Market Management System. The user updates different product details, including quantity, price, and expiry date, by selecting the appropriate option. The program modifies the corresponding records in the database and confirms each successful update, ensuring accurate and up-to-date inventory information.

```
Which Super Market operation do you want to perform(Enter the integer): 1

Enter the product ID: 11
Enter the quantity: 10
   ID | Name          |       Price | Quantity | E.C
=============================================================
   11 | Pencil Set    |       20.00 |       10 | No expiry date


Do you want to add another product(y,n): y

Enter the product ID: 11
Enter the quantity: 3
Product already added to the list

Do you want to increase the quantity of this product(y,n): y


Do you want to add another product(y,n): y

Enter the product ID: 11
Enter the quantity: 7
Product already added to the list

Do you want to increase the quantity of this product(y,n): y

Only 2 more items available


Do you want to add another product(y,n): y

Enter the product ID: 3
Enter the quantity: 5
   ID | Name          |       Price | Quantity | E.C
=============================================================
    3 | Bread         |       30.00 |        5 | Expired


Do you want to add another product(y,n): n

Final bill is:

   ID | Name          |        Price | Quantity | E.C
=============================================================
   11 | Pencil Set    |        20.00 |       15 | No expiry date
    3 | Bread         |        30.00 |        5 | Expired

Total Price: 450.00
```

```
mysql> select * from products;
+------------+--------------+-------------------------+--------------------------+-------------+
| Product_ID | Product_Name | Total_Quantity_Available | Individual_Product_Price | Expiry_Date |
+------------+--------------+-------------------------+--------------------------+-------------+
|          1 | Chocolate    |                      70 |                    50.00 | 2027-05-09  |
|          2 | Milk         |                      20 |                    45.50 | 2026-05-08  |
|          3 | Bread        |                      15 |                    30.00 | 2026-06-03  |
|          4 | Biscuits     |                      50 |                    20.00 | 2026-08-20  |
|          5 | Bottle       |                      30 |                   150.00 | NULL        |
|          6 | Plates       |                      40 |                    70.00 | NULL        |
|          7 | Toot Brush   |                      60 |                    40.00 | NULL        |
|          8 | Toot Paste   |                      60 |                    10.00 | NULL        |
|          9 | Cake         |                      40 |                    20.00 | 2026-09-26  |
|         10 | Juice        |                      20 |                    20.00 | 2026-10-02  |
|         11 | Pencil Set   |                      15 |                    20.00 | NULL        |
|         12 | Glue         |                       5 |                    20.00 | NULL        |
|         13 | Butter       |                     100 |                    60.00 | 2026-09-06  |
|         14 | Books        |                      40 |                   100.00 | NULL        |
+------------+--------------+-------------------------+--------------------------+-------------+
14 rows in set (0.00 sec)
```

```
mysql> select * from products;
+------------+--------------+-------------------------+--------------------------+-------------+
| Product_ID | Product_Name | Total_Quantity_Available | Individual_Product_Price | Expiry_Date |
+------------+--------------+-------------------------+--------------------------+-------------+
|          1 | Chocolate    |                      70 |                    50.00 | 2027-05-09  |
|          2 | Milk         |                      20 |                    45.50 | 2026-05-08  |
|          3 | Bread        |                      10 |                    30.00 | 2026-06-03  |
|          4 | Biscuits     |                      50 |                    20.00 | 2026-08-20  |
|          5 | Bottle       |                      30 |                   150.00 | NULL        |
|          6 | Plates       |                      40 |                    70.00 | NULL        |
|          7 | Toot Brush   |                      60 |                    40.00 | NULL        |
|          8 | Toot Paste   |                      60 |                    10.00 | NULL        |
|          9 | Cake         |                      40 |                    20.00 | 2026-09-26  |
|         10 | Juice        |                      20 |                    20.00 | 2026-10-02  |
|         11 | Pencil Set   |                       0 |                    20.00 | NULL        |
|         12 | Glue         |                       5 |                    20.00 | NULL        |
|         13 | Butter       |                     100 |                    60.00 | 2026-09-06  |
|         14 | Books        |                      40 |                   100.00 | NULL        |
+------------+--------------+-------------------------+--------------------------+-------------+
14 rows in set (0.00 sec)
```

Database Before and After billing the products

The output demonstrates the billing and sales feature of the Super Market Management System. The user selects products and quantities for purchase. The program prevents duplicate entries, allows quantity updates, checks stock availability, and generates a final bill showing purchased items, quantities, prices, expiry status, and the total amount payable.

```
Which Super Market operation do you want to perform(Enter the integer): 8

Thank you for using the Super Market Application

C:\Users\Menshafi>
```

The output demonstrates the exit feature of the Super Market Management System. When the user selects option 8, the program terminates the application safely and displays a thank-you message. This provides a proper ending to the session, ensuring that all operations are completed before the user exits the system.

❖BIBLIOGRAPHY

- Computer Science with Python by Sumita Arora
- https://www.google.com/
- https://github.com/Menshafi-2629/Python-Projects
- https://learnpython.org/

❖CONCLUSION

The Super Market Management System was successfully developed using Python and MySQL to simplify and automate various supermarket operations. The project effectively manages product information, billing, inventory records, product searches, expiry checking, and product updates through a database-driven approach.

One of the most important features of this project is its ability to display the expiry status of products during billing, helping

ensure that customers are informed whether a product is expired, not expired, or does not have an expiry date. This contributes to customer safety and responsible inventory management.

The project also incorporates a trial-and-error approach through extensive input validation and exception handling. As a result, the program does not terminate or break when invalid inputs or unexpected errors occur. Instead, it guides the user to provide correct information and continues execution smoothly, making the application more reliable and user-friendly.

Although the current version operates through a text-based interface in Python IDLE, it can be further enhanced by implementing a Graphical User Interface (GUI) for a more professional user experience.

Overall, this project demonstrates the practical application of Python programming, database management, and error-handling techniques in developing an efficient and dependable supermarket management solution.


THANK YOU
THANK YOU!
THANK YOU